

```
---
title: "WISpR"
author: "Nuray Sogunmez Erdogan"
date: "2024-05-03"
output: pdf_document
---
```

```
```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```
```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
```{r load libraries}
library(Seurat)
library(ggplot2)
library(dplyr)
library(Matrix)
library(tidyverse)
library(reshape2)
library(S4Vectors)
library(png)
library(RColorBrewer)
require(data.table)
library(usethis)
usethis::edit_r_environ() #increase memory if needed
setwd("/Path/to/Directory")
```
```

Load Datasets

```
```{r data, echo=FALSE}
data_dir1 <- "/path/to/data"
list.files(data_dir1) # Should show barcodes.tsv, genes.tsv, and
matrix.mtx
expression_matrix1 <- Read10X(data.dir = data_dir1)

coord_ST_mouse <- read.csv("/path/to/tissuePositions", header = FALSE) ##x-
y- coordinates of the spots
rownames(coord_ST_mouse) <- coord_ST_mouse$V1
coord_ST_mouse <- coord_ST_mouse[(coord_ST_mouse$V2=="1"),] ## use only the
spots under the tissue
```
```

Intersecting Data

```
```{r match the spot ids, echo=FALSE}
match_n <- match(colnames(expression_matrix1),rownames(coord_ST_mouse))
```

```

coord_ST_mouse_reor <- coord_ST_mouse[match_n,]

write_csv(cbind((as.data.frame(coord_ST_mouse_reor))), "/
tissue_positions_list_aligned.csv")
```

## Creating Object

```{Create object, echo=FALSE}
counts.meta_ST.mouse <- CreateSeuratObject(counts = expression_matrix1,
assay = "RNA")

counts.meta_ST.mouse <- AddMetaData(object = counts.meta_ST.mouse,
metadata = coord_ST_mouse$V5, col.name = "X_coord")
counts.meta_ST.mouse <- AddMetaData(object = counts.meta_ST.mouse,
metadata = coord_ST_mouse$V6, col.name = "Y_coord")
```

## Reading and preprocessing scRNA-Seq data

```{Read and preprocess scRNA-Seq data, echo=FALSE}
counts <- fread("/path/to/cnt_data.dat", check.names=FALSE)
header <- read.table(paste0("/path/to/cnt_data_col1.tsv"))
counts <- t(counts)

colnames(counts) <- header$V1
counts <- counts[-1,]

counts_meta <- read.table(paste0("/path/to/mta_data.tsv")) #read filtered
meta data
rownames(counts_meta) = counts_meta$V1
counts_meta <- dplyr::select(counts_meta, -1)

counts.metasc <- CreateSeuratObject(counts = counts)

saveRDS(counts.metasc, "/path/to/target/dir/seurat_counts.rds")

colnames(counts.metasc) == rownames(counts_meta)

inter <- intersect(rownames(counts.metasc),
rownames(counts.meta_ST.mouse))
counts.metasc0 <- counts.metasc[inter,]
spat.st2 <- counts.meta_ST.mouse[inter,]

cell_types <- as.factor(counts_meta$V2)

counts.metasc0@meta.data$seurat_clusters <- cell_types

counts.metasc0 <- SetIdent(counts.metasc0, value = "seurat_clusters")
Idents(counts.metasc0) <- factor(x = Idents(counts.metasc0), levels =
sort(levels(counts.metasc0)))
Idents(counts.metasc0)
```

## Initiate Python Environment

```{Initiate Python Env and process scRNA-Seq data, echo=FALSE}

```

```

Sys.setenv(RETICULATE_PYTHON = "/path/to/pythonenv/python")
library(reticulate)

reticulate::virtualenv_create("//path/to/pythonenv/envs/giotto_env/bin/",
python = "/path/to/pythonenv/envs/giotto_env/bin/pythonw")

reticulate::use_virtualenv("/path/to/pythonenv/envs/giotto_env/bin/")

py_available()

my_python_path = "/path/to/pythonenv/envs/giotto_env/bin/pythonw"

library(Giotto)
instrs = createGiottoInstructions(python_path = my_python_path)

sc_matrix<-counts.metasc0@assays$RNA@counts

sc_lable<-counts.metasc0$cellname %>% as.factor()

sc_matrix2 <- createGiottoObject(raw_exprs = sc_matrix,instructions =
instrs)
sc_matrix2norm <- normalizeGiotto(gobject = sc_matrix2)
sc_matrix2norm@cell_metadata$leiden_clus<-as.character(sc_lable)
sc_matrix2norm@cell_metadata$major_clus <- majors$Main

gini_markers_subclusters = findMarkers_one_vs_all(gobject =
sc_matrix2norm,
 method = 'gini',
 expression_values =
'normalized',
 cluster_column =
'leiden_clus')

topgenes_gini = gini_markers_subclusters[, head(.SD, 100), by = 'cluster']

sc_norm_exp <- 2^(sc_matrix2norm@norm_expr)-1

ExprSubset<-sc_norm_exp[as.character(topgenes_gini$genes),]

Sig<-NULL
for (i in as.character(unique(sc_lable))){
 Sig<-cbind(Sig,(apply(ExprSubset,1,function(y)
mean(y[which(sc_lable==i)]))))
}
colnames(Sig)<-unique(sc_lable)
...

Processing ST Data

```{Process ST data, echo=FALSE}

st_matrix<-spat.st2@assays$RNA@counts %>% as.matrix()
st_matrix <- as.data.frame(st_matrix)
st_matrix2 <- st_matrix
sort(colSums(st_matrix2), decreasing = FALSE)
#st_matrix2 = dplyr::select(st_matrix, -c(`16.85x24.93`))

```

```

st_matrix2 <- createGiottoObject(raw_exprs = st_matrix2, instructions =
instrs)
st_matrix2norm <- normalizeGiotto(gobject = st_matrix2)
all(is.finite(st_matrix2norm@norm_expr)) #to make sure no NA values
introduced

spot_giotto <- calculateHVG(gobject = st_matrix2norm)
gene_metadata = fDataDT(spot_giotto)
featgenes = gene_metadata[hvg == 'yes']$gene_ID
spot_giotto <- runPCA(gobject = spot_giotto, genes_to_use = featgenes,
scale_unit = F)
signPCA(spot_giotto, genes_to_use = featgenes, scale_unit = F)
spot_giotto <- createNearestNetwork(gobject = spot_giotto,
dimensions_to_use = 1:10, k = 10)
library(igraph)
spot_giotto <- Giotto::doLeidenCluster(gobject = spot_giotto, resolution =
0.2, n_iterations = 1000)
gini_markers_subclusters_st = findMarkers_one_vs_all(gobject =
spot_giotto,
method = 'gini',
expression_values =
'normalized',
cluster_column =
'leiden_clus')

topgenes_gini_st = gini_markers_subclusters_st[, head(.SD, 100), by =
'cluster']
st_norm_exp_st <- 2^(st_matrix2norm@norm_expr)-1
ExprSubset_st<-st_norm_exp_st[as.character(topgenes_gini_st$genes),]
st_label <- spot_giotto@cell_metadata$leiden_clus
allmarkers_unique <- unique(cbind(c(topgenes_gini$genes,
topgenes_gini_st$genes)))

ExprSubset_st2<-st_norm_exp_st[as.character(allmarkers_unique),]

Sig_st2<-NULL
for (i in
as.character(unique(unique(spot_giotto@cell_metadata$leiden_clus)))){
  Sig_st2<-cbind(Sig_st2,(apply(ExprSubset_st2,1,function(y)
mean(y[which(st_label==i)]))))
}

spat2 <- spat.st2[allmarkers_unique, ]

sc_matrix_2 <- counts.metasc0[allmarkers_unique, ]
counts.metasc0@meta.data

AE <- AverageExpression(object =sc_matrix_2, features = allmarkers_unique,
return.seurat = FALSE, group.by = "cellname", slot
= "counts", verbose = TRUE)

match <- match(rownames(AE$RNA),rownames(spat2))
spat.st2_AE25 <- spat2[match,]

```

```
write_csv(cbind(rownames(AE$RNA), as.data.frame(AE$RNA)), "path/to/save/
WISpR_sc.dat")
write_csv(cbind(rownames(spat.st2_AE25@assays$RNA@counts),
(as.data.frame(spat.st2_AE25@assays$RNA@counts))), "/path/to/save/
WISpR_ST.dat")
` ``
```